

Microservicios en Netflix: análisis de un caso real

Sánchez Pilaloe Andy Paul
Aplicaciones Distribuidas – Ingeniería en Software

Estilo seleccionado: arquitectura de microservicios. Netflix es un caso representativo de adopción de microservicios en producción, porque pasó de depender de una infraestructura más centralizada a operar una plataforma distribuida sobre la nube. De acuerdo con su propio reporte técnico, uno de los hechos que impulsó este cambio fue una falla grave de base de datos ocurrida en 2008, la cual afectó durante varios días la operación del servicio de DVD. A partir de esa experiencia, la empresa inició una migración progresiva hacia AWS y organizó su sistema en servicios independientes, capaces de evolucionar sin detener toda la plataforma [1].

La elección de microservicios tuvo sentido porque Netflix necesitaba atender millones de usuarios, distintos dispositivos y cargas variables según región, horario o estreno de contenido. En una arquitectura monolítica, un cambio en una parte del sistema puede afectar a toda la aplicación; en cambio, con microservicios cada módulo puede desarrollarse, probarse y desplegarse con mayor independencia. La literatura señala que este estilo favorece la escalabilidad, la autonomía de equipos y el despliegue frecuente, aunque también exige mayor control sobre comunicación, monitoreo y pruebas distribuidas [2].

Los principales beneficios observados en este caso fueron la elasticidad, la disponibilidad y la capacidad de innovación continua. Si un servicio específico falla, el resto de la plataforma puede seguir funcionando o degradarse de forma parcial. Además, cada componente puede escalar según su propia demanda, sin obligar a aumentar recursos para todo el sistema. No obstante, esta decisión también trajo desafíos concretos: mayor complejidad operativa, necesidad de observabilidad, manejo de latencia entre servicios, control de versiones de APIs y diseño cuidadoso para tolerar fallos. Para ello, Netflix desarrolló herramientas propias como *Hystrix* (tolerancia a fallos mediante circuit breakers) y *Zuul* (API Gateway para enrutamiento y seguridad centralizada). Por ello, los microservicios no deben verse como una solución universal, sino como una alternativa adecuada cuando el sistema requiere crecimiento, cambios frecuentes y separación clara de responsabilidades [3].

En el Proyecto Fin de Curso, orientado a un *e-commerce de componentes de PC con módulo de ensamblado*, este caso sirve como referencia para dividir el sistema en módulos independientes: usuarios, catálogo, inventario, pedidos, pagos, recomendaciones y compatibilidad de componentes. El módulo de ensamblado podría operar como un servicio propio que consulte información de CPU, GPU, placa madre, memoria y fuente de poder mediante APIs internas. Esta separación permitiría escalar las partes más usadas durante promociones, aislar fallos y facilitar futuras funciones de inteligencia artificial, como la detección de cuellos de botella entre CPU y GPU. No obstante, para evitar una sobrearquitectura, el proyecto debería iniciar con pocos servicios bien definidos y un API Gateway sencillo, priorizando claridad, mantenibilidad y documentación OpenAPI.

Referencias

- [1] Netflix Technology Blog. “Completing the Netflix Cloud Migration,” visitado 3 de jun. de 2026. dirección: <https://about.netflix.com/news/completing-the-netflix-cloud-migration>.
- [2] I. K. Aksakalli, T. Celik, A. B. Can y B. Tekinerdogan, “Deployment and communication patterns in microservice architectures: A systematic literature review,” *Journal of Systems and Software*, vol. 180, pág. 111 014, 2021. DOI: 10.1016/j.jss.2021.111014.
- [3] G. Blinowski, A. Ojdowska y A. Przybylek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803.